Ch 3 Problems with file Based system

Authored by : Arpit Raj , Lnmiit jaipur

1. Data redundancy

some info across multiple storage location

causes → Storage overhead

↓

Increase Database size
Increase memory usage

Maintenance overhead

↓

one update needs to update same thing in all other files

2. Data inconsistency

multiple copies of same data contain different values

Eg: customers.csv	orders.csv
9921 Aadya	9921 Aadz

`Redundancy → multiple copies → partial update → inconsistency`

3. Data isolation

```
→ student.csv
→ hostel.csv
→ attendance.csv
```

Show all students with >90% attendance

→ **This needs** → read every file, parse every record, perform joins manually, filter results

> DBMS lets you to express this with a SQL query and optimizer defines an efficient execution strategy

4. Difficult data access

→ write the search logic yourself, DBMS handles optimization and execution

5. Data integrity issues

→ invalid data is also allowed, a DBMS prevents this using foreign key constraints

6. Atomicity issues

→ no guarantee that related operations complete as a single unit, DBMS solves this with transactions

7. Concurrent Access issues

8. Security issues

→ file system gives only coarse grained permissions (typically at file level)

> DBMS supports fine grained permissions

9. Recovery issues

→ no concept of recovery logs and procedures to restore the database to a consistent state in case server crashes while writing

Practice Questions

Q1. Why does data redundancy increase the risk of inconsistency?

> **A1.** Data redundancy creates multiple copies of the same logical information. If an update is applied to only some of those copies, they diverge, leading to data inconsistency. Thus, redundancy increases both the likelihood and maintenance cost of keeping data synchronized.

Q2. Explain program-data dependence with an example.

> **A2.** Program-data dependence means application logic is tightly coupled to the physical structure of data files. For example, if an application expects a CSV with columns (ID, Name, Age) and the file changes to (ID, Name, DOB), the parsing logic breaks and the application must be modified. A DBMS reduces this coupling through schema abstraction and data independence.

Q3. Why is data isolation a problem in file-based systems?

> **A3.** In file-based systems, related data is scattered across multiple independent files. Applications must manually combine, filter, and maintain relationships between these files, increasing code complexity, reducing maintainability, and making complex queries inefficient.

Q4. What is a lost update? How does it occur?

> **A4.** A lost update occurs when two concurrent operations read the same data, modify it independently, and one update overwrites the other. This happens because there is no proper concurrency control to coordinate simultaneous writes.

Q5. Why can't a file system enforce referential integrity?

> **A5.** A file system stores bytes and has no understanding of relationships between records. It cannot verify that a CustomerID in one file actually exists in another. Referential integrity requires semantic knowledge of the data, which is provided by a DBMS through foreign key constraints.

Q6. Explain atomicity using a banking example.

> **A6.** In a bank transfer, debiting one account and crediting another must be treated as a single transaction. If the system crashes after the debit but before the credit, money appears to disappear. Atomicity guarantees that either both operations succeed or both are rolled back.

Q7. Compare file-level security with database-level security.

> **A7.** File systems generally provide coarse-grained permissions at the file or directory level (e.g., read, write, execute). DBMSs support fine-grained authorization, allowing permissions at the database, table, column, and sometimes row level, along with roles, auditing, and privilege management.

Q8. Which DBMS features directly address the major problems of file-based systems?

> **A8.** Redundancy → Normalization
> Inconsistency → Constraints + Transactions
> Program-Data Dependence → Data Independence
> Difficult Access → SQL + Query Optimizer
> Integrity Issues → Primary/Foreign Keys, CHECK Constraints
> Atomicity Problems → ACID Transactions
> Concurrency Problems → Locks, MVCC, Isolation Levels
> Security Problems → Authentication, Authorization, Roles
> Recovery Problems → Write-Ahead Logging, Checkpoints, Recovery Manager